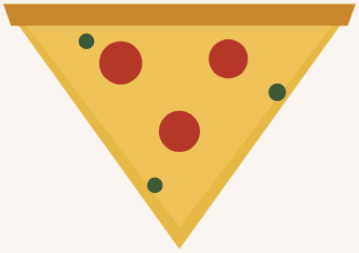
PROYECTO DE PROGRAMACIÓN · MEMORIA

Caja registradora de una pizzería

Desarrollado en Python





INTEGRANTES

Alex Rodríguez
Pau Mas

FECHA
Octubre de 2026

HERRAMIENTAS
Python · GitHub · Trello

REPOSITORIO
Puuaa7/ProyectoCajaRegistradora

Memoria del proyecto PIZZA

Proyecto de Programación · Oct 22, 2026

1. Integrantes y datos del proyecto

Integrante	Usuario en GitHub	Iniciales en Trello
Alex Rodríguez	alexrb29	AR
Pau Mas	Puuaa7	PR

- **Proyecto:** caja registradora de una pizzería en Python.
- **Repositorio:** github.com/Puuaa7/ProyectoCajaRegistradora
- **Tablero:** Trello, «Caja Registradora - Pizzería».
- **Link Trello:** https://trello.com/b/nt1k8GgM/%F0%9F%8D%95-caja-registradora-pizzeria

2. Distribución de tareas y responsabilidades

Nos repartimos el trabajo por partes del programa: Pau se encargó sobre todo de los cálculos, las ofertas y el main. Alex del carrito, el ticket, las bebidas, las pruebas y el README. Cada tarea es una tarjeta de Trello con responsable, descripción, lista de comprobación y estado.

"Las tareas que ponemos pendientes es porque a día de hoy 22/10/26 que estamos trabajando en ellas y avanzando en la memoria a la vez, el día de la entrega ya estarán todas terminadas"

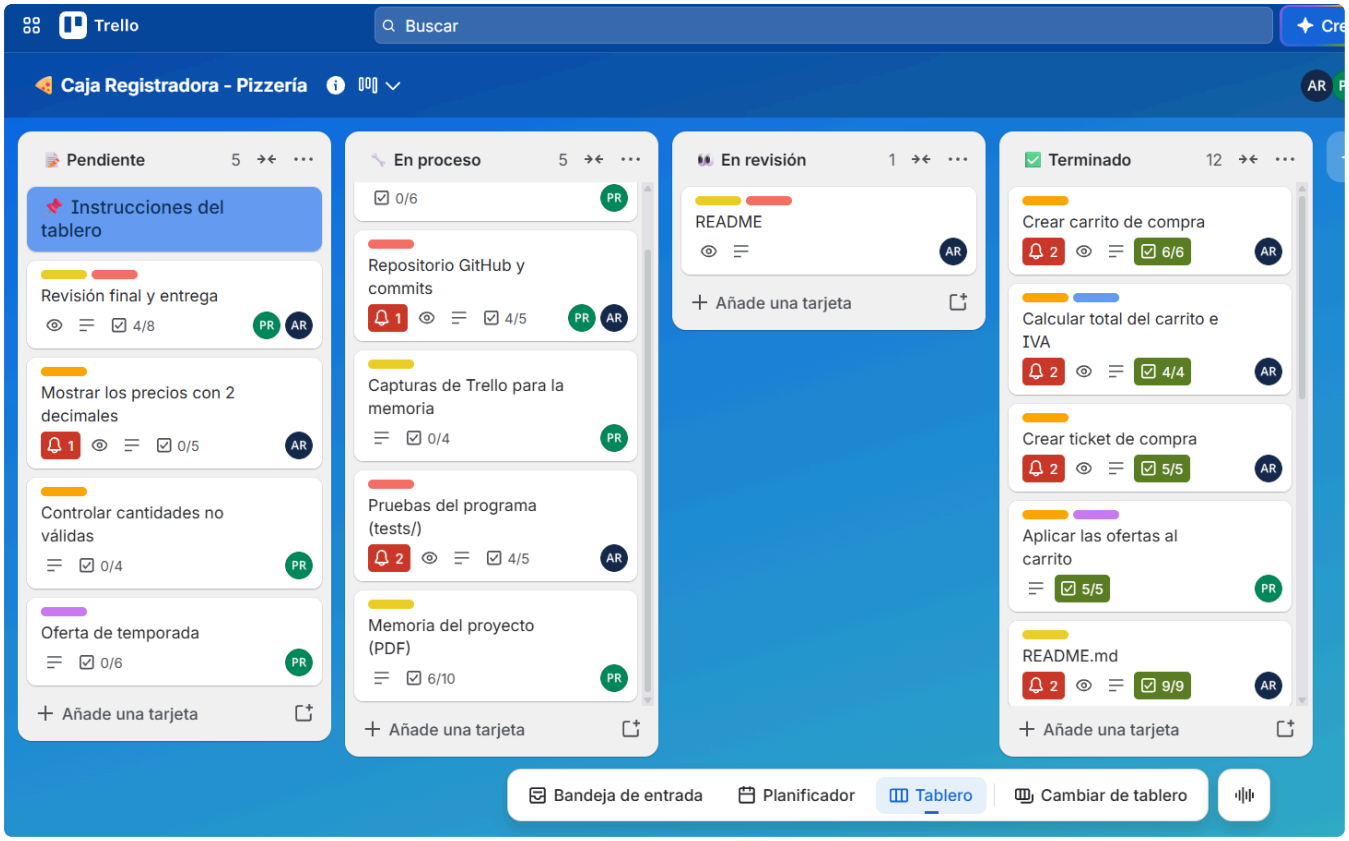
Tarea (tarjeta de Trello)	Responsable	Estado a 22/10
Revisar gastos fijos y temporadas	Pau	Terminado
Comprobar cálculo de precios e IVA	Pau	Terminado
Programar 4 ofertas	Pau	Terminado
Aplicar las ofertas al carrito	Pau	Terminado
Crear bebidas y precios	Alex	Terminado
Crear carrito de compra	Alex	Terminado

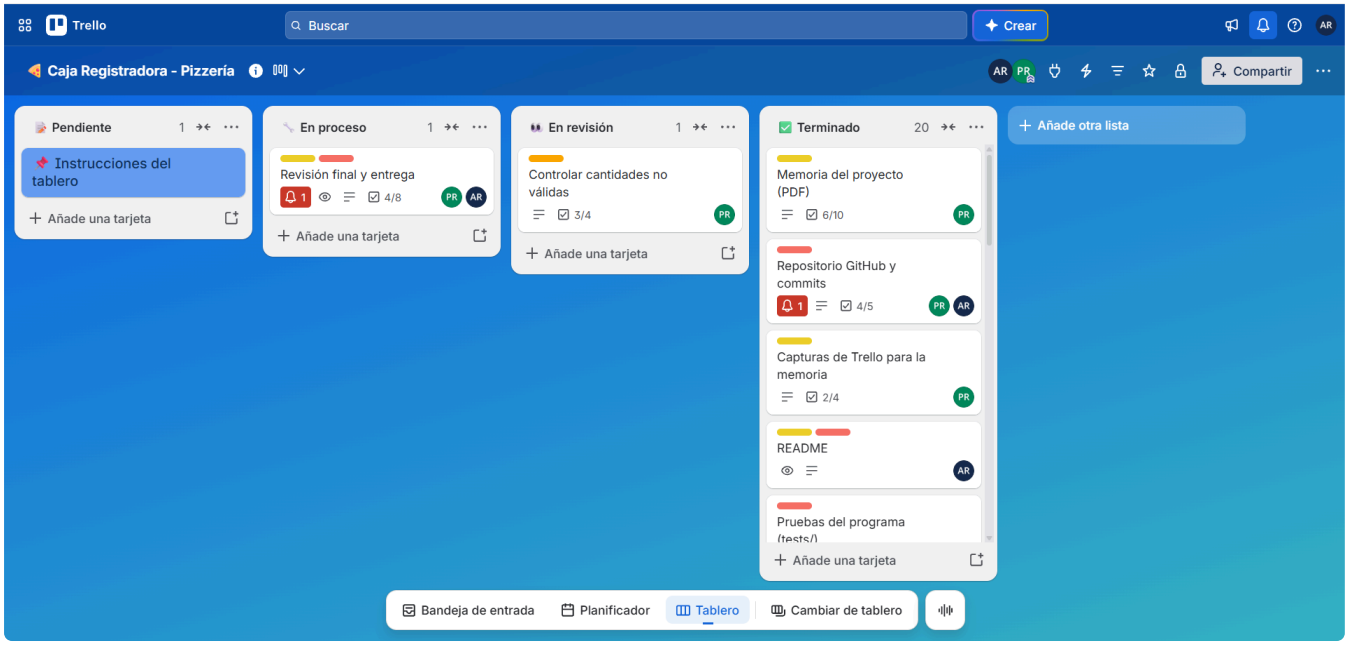
Tarea (tarjeta de Trello)	Responsable	Estado a 22/10
Calcular total del carrito e IVA	Alex	Terminado
Crear ticket de compra	Alex	Terminado
README.md	Alex	Terminado
README (cambios del 22/10)	Alex	En revisión
Cálculo de precio por temporada	Pau	En proceso
Pruebas del programa (tests/)	Alex	En proceso
Capturas de Trello para la memoria	Pau	En proceso
Memoria del proyecto (PDF)	Pau	En proceso
Repositorio GitHub y commits	Los dos	En proceso
Mostrar los precios con 2 decimales	Alex	Pendiente
Controlar cantidades no válidas	Pau	Pendiente
Oferta de temporada	Pau	Pendiente
Revisión final y entrega	Los dos	Pendiente

En GitHub cada parte tiene su rama (`feature/calculos`, `feature/pizzas`, `feature/ofertas`, `feature/aplicar-ofertas`, `feature/bebidas`, `feature/ticket`, `feature/menu`, `test/pruebas`, `docs/readme` y `docs/memoria`), que se une a `main` con un merge.

3. Tablero de Trello

El tablero tiene cuatro columnas: Pendiente, En proceso, En revisión y Terminado. Cada tarjeta lleva una etiqueta de color por tipo de tarea, la lista de pasos y la persona responsable (AR = Alex, PR = Pau).





4. Reuniones y acuerdos

Nos reunimos en cuatro sesiones de trabajo, que coinciden con los días en que hay commits en GitHub.

Fecha	Qué se trabajó	Acuerdos
22/10/2026	Cambios en el README y memoria	Cada cambio nuevo va en su propia rama y se sube en cuanto está terminado.
21/10/2026	Pasar el código a ramas y unirlas a `main`	Una rama por parte del programa, con su responsable de Trello. Unir primero cálculos, pizzas, carrito y ofertas, y después bebidas, ticket, menú, pruebas y README.
14/10/2026	Reorganizar el proyecto en `src/`, `docs/` y `tests/`; bebidas, carrito y oferta 2x1	Usar la estructura `src/` y guardar el código antiguo en `z.functions_antiguo`.
07/10/2026	Estructura inicial, ingredientes, pizzas y gastos	Calcular el coste de cada pizza con el precio por kg de cada ingrediente y los gramos que lleva. Repartir las tareas en Trello.

5. Decisiones importantes

La decisión principal fue calcular el precio de cada pizza a partir de sus ingredientes reales, en lugar de inventar precios de carta.

Decisión	Motivo
Precio de los ingredientes por kg y gramos de cada ingrediente por pizza	Así el coste depende de lo que lleva cada pizza de verdad.
21 pizzas en un diccionario de diccionarios (`pizza → ingrediente → gramos`)	El enunciado pide al menos 20 y así se recorren fácil con un bucle.
Margen de beneficio del 35% (multiplicar por 1,35)	Es el del enunciado y da precios de entre 11,95 y 17,36 € en temporada media.
Bebidas con precio fijo de carta, IVA incluido	Solo se compran y se venden.
El IVA se separa al final, sobre el total con descuentos	Así el IVA se calcula sobre lo que paga de verdad el cliente.
Estructura de carpetas `src/`, `docs/` y `tests/`	Es más ordenada y permite importar con `from src...`.

Decisión	Motivo
El día de la semana se coge de la fecha del ordenador	Para que la oferta del martes funcione sola; se puede cambiar en el menú para probarla.
Usar listas, diccionarios, tuplas y conjuntos	Carrito = lista, carta = diccionario, temporadas y días = tuplas, pizzas del 2x1 = conjunto.

6. Errores encontrados y cómo se resolvieron

Error	Causa	Solución
`calcularPrecioPizza` fallaba al ejecutarse	Usaba una variable `total` que no existía	Usar la variable correcta, `precioTotalPizza`.
`ModuleNotFoundError: No module named 'Ofertas'`	Se ejecutaba `ticket.py` en vez del archivo principal	Ejecutar siempre `main.py`, que es el que arranca el programa.
`main.py` salía vacío en VS Code	Había una pestaña abierta con cambios sin guardar de una versión antigua	Cerrar la pestaña sin guardar y volver a abrir el archivo.
El 2x1 solo funcionaba con dos pizzas iguales	Se contaba cada pizza por separado	Juntar los precios de todas las pizzas de la oferta, ordenarlos y regalar la más barata de cada pareja.
«margarita » con un espacio al final no se encontraba	No se quitaban los espacios	Añadir `.strip()` al buscar el producto.
Las ofertas no se veían en el carrito, solo en el ticket	El carrito mostraba solo el total sin ofertas	Nueva función `mostrarDescuentosCarrito`, usada en el carrito y en «Ver ofertas».
La carpeta del proyecto no estaba conectada a Git	Se había descargado de GitHub como ZIP	Clonar el repositorio con `git clone` y trabajar siempre en esa carpeta.
Al unir ramas, `main` dejaba de funcionar	El menú y el ticket usaban funciones de ramas que aún no estaban unidas	Unir las ramas en orden: primero las de las que dependen las demás.

Error	Causa	Solución
Las cinco ramas de Alex se subieron a GitHub a la vez	Se preparó todo fuera del repositorio y se subió junto con un script	Desde entonces, cada cambio se sube en cuanto está terminado (ver apartado 8).

7. Cambios de criterio respecto a la idea inicial

Antes	Después	Por qué
Carpetas `1.Main`, `2.Functions` y `3.SistemasDeOfertas`	Carpetas `src/`, `docs/` y `tests/`	Era más ordenado y evitaba añadir rutas a mano con `sys.path`.
8 pizzas	21 pizzas	El enunciado pide un mínimo de 20.
2x1 en Margarita, Prosciutto y Barbacoa	2x1 en Margarita, Prosciutto y Hawaiana, que se pueden mezclar	Es más justo para el cliente y más fácil de entender.
2 ofertas (2x1 y 10% en pedidos de más de 30 €)	4 ofertas: se añaden menú pizza + bebida y 20% en pizzas los martes	El enunciado pide al menos 3.
El programa preguntaba qué día era	Coge el día de hoy automáticamente	Era más cómodo; se puede cambiar desde el menú.
Ticket con columnas alineadas y fecha	Ticket sencillo: productos, descuentos, base imponible, IVA y total	Se parece más al ejemplo del enunciado y es más fácil de leer.

8. Problemas de coordinación y cómo se solucionaron

1. **Dos versiones del proyecto a la vez.** Alex: estaba trabajaba con la estructura de carpetas antigua mientras que Pau reorganizaba el repositorio en `src/`. Lo solucionamos adaptando el código de Alex a la nueva estructura y manteniendo los nombres de funciones de Pau.
2. **Subida de las ramas de golpe.** Al pasar su parte al repositorio, Alex subió las cinco ramas (`feature/bebidas`, `feature/ticket`, `feature/menu`, `test/pruebas` y `docs/readme`) a la vez el 21/10, en lugar de una a una. Luego Pau las unió todas a `main`

ese mismo día. No se puede deshacer porque GitHub guarda la actividad, así que lo explicamos aquí.

3. **Orden de los merges.** Algunas partes dependían de otras (el ticket necesita las ofertas y los cálculos). Acordamos un orden y comprobamos que el programa funcionaba después de unir todo.

Desde el 22/10 trabajamos así: `git pull` en `main`, una rama nueva por cada cambio, commits pequeños, `git push` al terminar y un merge para unirlo a `main`.

9. Cálculo económico: breve explicación del cálculo de precios

El precio de una pizza sale de sumar sus costes y aplicarle un 35% de beneficio y un 10% de IVA. Una Margarita cuesta 11,95 € en temporada media.

```
\text{precio} = (\text{ingredientes} + 0{,}60 + \frac{4200 + 1600}
{\text{pizzas al mes}}) \times 1{,}35 \times 1{,}10
```

1. **Ingredientes:** precio por kg × gramos / 1000, sumando todos los de la pizza.
2. **Suministros:** 0,60 € por pizza (luz, gas, agua, horno y embalaje).
3. **Gastos fijos:** personal (4200 €) + alquiler (1600 €) al mes, repartidos entre las pizzas que se esperan vender.
4. **Beneficio:** el coste total se multiplica por 1,35.
5. **IVA:** el precio sin IVA se multiplica por 1,10.

Desglose de la Margarita (250 g de harina, 100 g de tomate y 120 g de mozzarella) en temporada media:

Concepto	Importe (€)
Ingredientes	1,65
Suministros	0,60
Gastos fijos (5800 / 1000)	5,80
Coste total	**8,05**
Beneficio (35%)	2,82
Precio sin IVA	**10,87**
IVA (10%)	1,08
Precio de venta	**11,95**

Los gastos fijos pesan mucho, así que el precio cambia según la temporada:

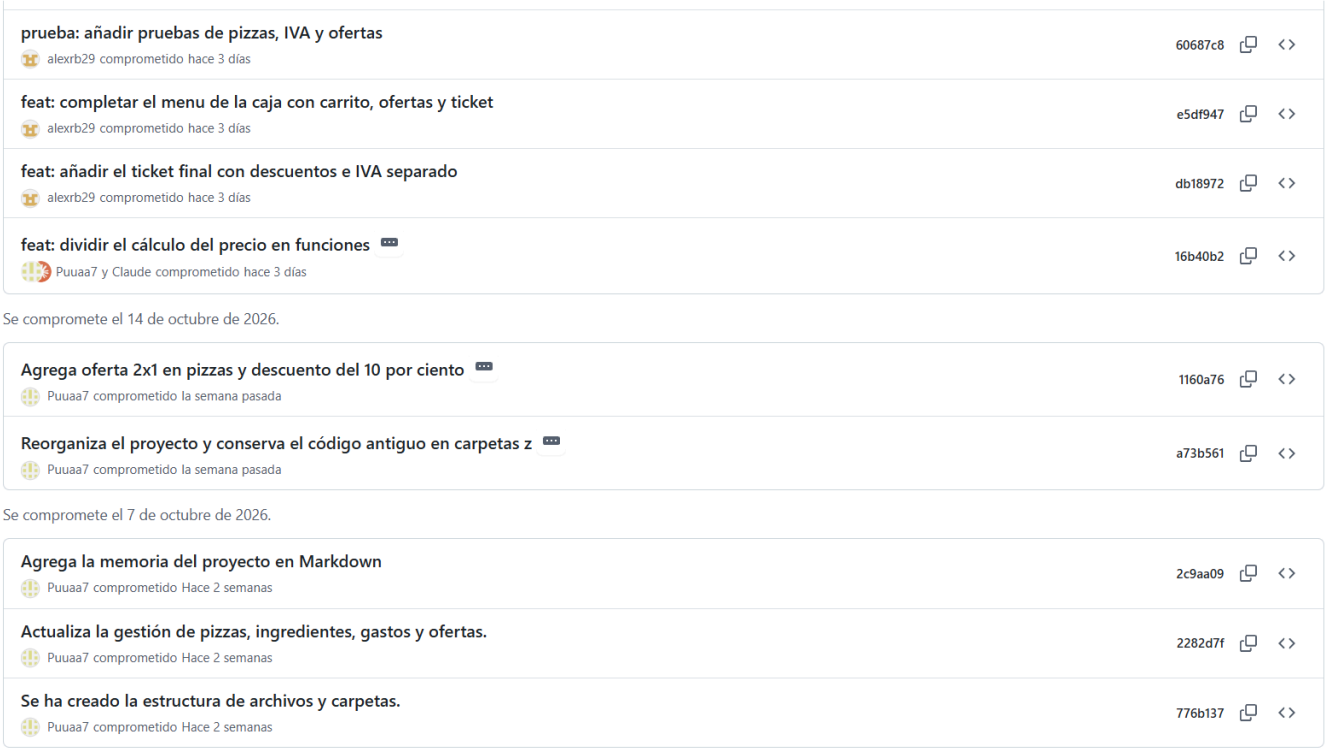
Temporada	Pizzas al mes	Gastos fijos por pizza (€)	Precio de la Margarita (€)
Baja	600	9,67	17,70
Media	1000	5,80	11,95
Alta	1500	3,87	9,08

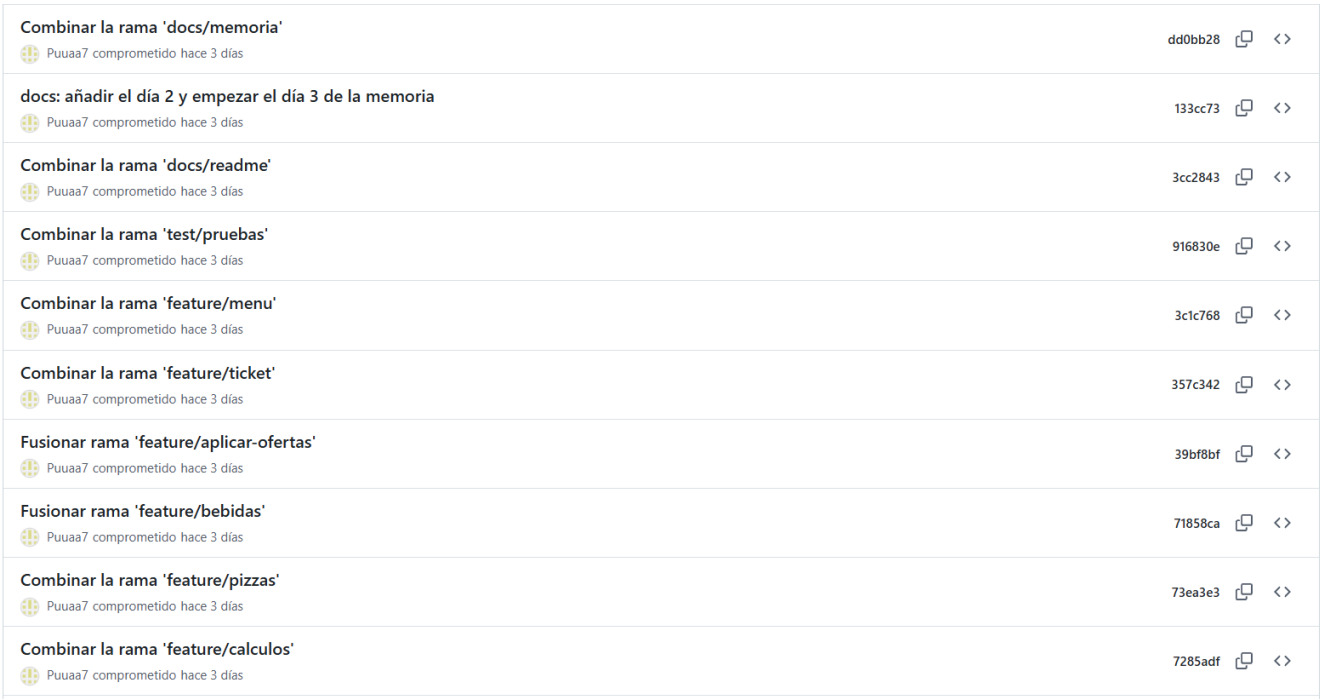
IVA en el ticket. Los precios de la carta ya llevan IVA. Al pagar se suman los productos, se restan los descuentos y del total se separa el IVA: base imponible = total / 1,10 e IVA = total − base imponible.

Bebidas. Tienen un precio fijo con IVA incluido (agua 1,20 €, refrescos 2,00 €, té frío 2,20 €, cerveza 2,50 €), porque no se cocinan.

10. Captura del historial de commits

El historial completo está en github.com/Puuaa7/ProyectoCajaRegistradora/commits/main. Cada parte del programa se hizo en su rama y se unió a `main` con un merge.







11. Conclusión

El programa cumple lo que pide el enunciado: 21 pizzas con su precio calculado, bebidas, carrito, 4 ofertas y un ticket con el IVA separado. Las 7 pruebas pasan correctamente.

Lo que mejor ha funcionado ha sido repartir las tareas en Trello por partes del programa, porque cada uno sabía qué le tocaba. Lo que peor ha funcionado ha sido Git: trabajamos demasiado tiempo fuera del repositorio y al final subimos y unimos muchas cosas a la vez. También durante una clase de proyecto trabajamos con live share en vez de con ramas, eso dificultó mucho hacer los comits por separado, solo los hacía Pau.

La lección principal es que hay que hacer commits y crear las ramas desde el primer día, y subir cada parte en cuanto está terminada. Arreglarlo al final nos costó más que hacerlo bien desde el principio.